# Programming Assignment #3

Name: Amna Malik ID: R11827604

## Problem-1

*(Your Codes)*

```
package parts.part1;

import java.util.Scanner;

/**
 * DriverExam models a 20-question driver's license exam.
 * It stores correct answers, student answers, grading results,
 * and provides methods to report pass/fail and missed questions.
 */
class DriverExam {

    // Stores the correct answers for the 20-question exam.
    private char[] correctAnswers;

    // Stores the student's answers for the 20-question exam.
    private char[] givenAnswers;

    // Stores the number of questions the student answered correctly.
    private int totalCorrect;

    // Stores which questions were missed (true means missed).
    private boolean[] missedQuestions;

    // Tracks whether grading has been performed to avoid re-grading.
    private boolean isGraded;

    /**
     * Constructs a DriverExam with the student's answers.
     * Parameters:
     * - studentAnswers: a char array of length 20 containing answers A-D
     * Intended use:
     * - Initializes the exam with the provided student answers so it can be
graded.
     */
    public DriverExam(char[] studentAnswers) {

        // Initializes the correct answers array with 20 entries.
        correctAnswers = new char[] {
                'B', 'D', 'A', 'A', 'C',
                'A', 'B', 'A', 'C', 'D',
                'B', 'C', 'D', 'A', 'D',
                'C', 'C', 'B', 'D', 'A'
        };

        // Stores the student's answers.
        givenAnswers = studentAnswers;
```

```
        // Initializes total correct to 0 before grading.
        totalCorrect = 0;

        // Initializes the missed questions tracking array to length 20.
        missedQuestions = new boolean[20];

        // Marks the exam as not yet graded.
        isGraded = false;
    }

    /**
     * Determines whether the student passed the exam.
     * Parameters:
     * - none
     * Intended use:
     * - Grades the exam if needed, then returns true if totalCorrect >= 15,
else false.
     */
    public boolean passed() {

        // Ensures grading is done before checking pass/fail.
        gradeExam();

        // Returns true if the student got at least 15 correct answers.
        return totalCorrect >= 15;
    }

    /**
     * Returns the total number of correct answers.
     * Parameters:
     * - none
     * Intended use:
     * - Grades the exam if needed, then returns the count of correct
answers.
     */
    public int totalCorrect() {

        // Ensures grading is done before returning correct count.
        gradeExam();

        // Returns the total correct.
        return totalCorrect;
    }

    /**
     * Returns the total number of incorrect answers.
     * Parameters:
     * - none
     * Intended use:
     * - Grades the exam if needed, then returns 20 - totalCorrect.
     */
    public int totalIncorrect() {

        // Ensures grading is done before returning incorrect count.
        gradeExam();

        // Returns the total incorrect.
```

```
        return 20 - totalCorrect;
    }

    /**
     * Returns the question numbers that were missed as a comma-separated
String.
     * Parameters:
     * - none
     * Intended use:
     * - Grades the exam if needed, then reports which question numbers were
missed.
     */
    public String questionsMissed() {

        // Ensures grading is done before building missed list.
        gradeExam();

        // missedList builds a string listing missed question numbers.
        StringBuilder missedList = new StringBuilder(); // missedList builds
the missed question output.

        // first tracks if we are adding the first missed question (comma
formatting).
        boolean first = true; // first is used to format commas properly.

        // Loops through the missedQuestions array.
        for (int i = 0; i < missedQuestions.length; i++) {
            // Checks if question i was missed.
            if (missedQuestions[i]) {
                // Adds comma and space if not the first missed question.
                if (!first) {
                    missedList.append(", ");
                }

                // Appends the question number (i + 1) because questions are
1-based.
                missedList.append(i + 1);

                // After adding the first item, mark first as false.
                first = false;
            }
        }

        // If nothing was missed, return "None".
        if (missedList.length() == 0) {
            return "None";
        }

        // Returns the built list.
        return missedList.toString();
    }

    /**
     * Grades the exam by comparing givenAnswers to correctAnswers.
     * Parameters:
     * - none
     * Intended use:
```

```
     * - Updates totalCorrect and missedQuestions exactly once.
     */
    private void gradeExam() {

        // If already graded, do nothing.
        if (isGraded) {
            return;
        }

        // Resets totalCorrect in case of any future extensions.
        totalCorrect = 0;

        // Compares each answer.
        for (int i = 0; i < correctAnswers.length; i++) {
            // Checks if the student's answer matches the correct answer.
            if (givenAnswers[i] == correctAnswers[i]) {
                // Increments correct count if matched.
                totalCorrect++;

                // Marks question as not missed.
                missedQuestions[i] = false;
            } else {
                // Marks question as missed.
                missedQuestions[i] = true;
            }
        }

        // Marks grading as complete.
        isGraded = true;
    }
}

/**
 * DriverExamDemo prompts the user for answers to a driver's license exam,
 * validates input, and prints pass/fail results and missed questions.
 */
public class DriverExamDemo {

    /**
     * Program entry point.
     * Parameters:
     * - args: command-line arguments (unused)
     * Intended use:
     * - Collects 20 validated answers, creates a DriverExam, and prints
results.
     */
    public static void main(String[] args) {

        // input reads user input from the keyboard.
        Scanner input = new Scanner(System.in); // input is used to read
keyboard input.

        // studentAnswers stores the student's answers for all 20 questions.
        char[] studentAnswers = new char[20]; // studentAnswers stores the
validated exam answers.

        // Loops through questions 1 to 20.
```

```
        for (int i = 0; i < studentAnswers.length; i++) {

            // valid stores whether the current answer is valid.
            boolean valid = false; // valid controls the input validation
loop.

            // Re-prompts until a valid answer is entered.
            while (!valid) {

                // Prompts the user for the current question answer.
                System.out.print("Question " + (i + 1) + " Student Answer:
");

                // line stores the raw input line.
                String line = input.nextLine(); // line stores the user's
typed response.

                // Trims whitespace and converts to uppercase to standardize.
                line = line.trim().toUpperCase();

                // Validates that input is exactly one character and is A-D.
                if (line.length() == 1) {

                    // ans stores the single character answer entered.
                    char ans = line.charAt(0); // ans stores the validated
letter candidate.

                    // Checks if the answer is one of A, B, C, D.
                    if (ans == 'A' || ans == 'B' || ans == 'C' || ans == 'D')
{

                        // Stores the valid answer.
                        studentAnswers[i] = ans;

                        // Marks as valid to exit loop.
                        valid = true;

                    } else {
                        // Prints validation error when not A-D.
                        System.out.println("Answer Validation - Please enter
A, B, C, or D.");
                    }

                } else {
                    // Prints validation error when not a single character.
                    System.out.println("Answer Validation - Please enter a
single letter A, B, C, or D.");
                }
            }
        }

        // exam creates a DriverExam object using collected answers.
        DriverExam exam = new DriverExam(studentAnswers); // exam grades and
reports exam results.

        // Prints whether the student passed.
        System.out.println("Passed: " + exam.passed());
```

```
        // Prints total correct.
        System.out.println("Total Correct: " + exam.totalCorrect());

        // Prints total incorrect.
        System.out.println("Total Incorrect: " + exam.totalIncorrect());

        // Prints the question numbers missed.
        System.out.println("Questions Numbers Missed: " +
exam.questionsMissed());

        // Closes the scanner.
        input.close();
    }
}
```

---

*(Output- screenshots)*

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-java
Question 1 Student Answer: b
Question 2 Student Answer: a
Question 3 Student Answer: a
Question 4 Student Answer: d
Question 5 Student Answer: c
Question 6 Student Answer: a
Question 7 Student Answer: d
Question 8 Student Answer: a
Question 9 Student Answer: c
Question 10 Student Answer: a
Question 11 Student Answer: b
Question 12 Student Answer: c
Question 13 Student Answer: d
Question 14 Student Answer: a
Question 15 Student Answer: d
Question 16 Student Answer: c
Question 17 Student Answer: c
Question 18 Student Answer: d
Question 19 Student Answer: d
Question 20 Student Answer: a
Passed: true
Total Correct: 15
Total Incorrect: 5
Questions Numbers Missed: 2, 4, 7, 10, 18

Process finished with exit code 0
```

## Problem-2

*(Your Codes)*

```
package parts.part2;

import java.util.Scanner;

/**
 * ChargeAccountValidator validates charge account numbers by comparing
 * a provided number against a list of known valid account numbers.
 */
class ChargeAccountValidator {

    // Stores the list of valid charge account numbers.
    private int[] validAccounts;

    /**
     * Constructs a ChargeAccountValidator and loads the valid account list.
     * Parameters:
     * - none
     * Intended use:
     * - Initializes the validator with the required set of valid account
numbers.
     */
    public ChargeAccountValidator() {

        // Initializes the array of valid charge account numbers.
        validAccounts = new int[] {
                5658845, 4520125, 7895122, 8777541, 8451277, 1302850,
                8080152, 4562555, 5552012, 5050552, 7825877, 1250255,
                1005231, 6545231, 3852085, 7576651, 7881200, 4581002
        };
    }

    /**
     * Determines whether an account number is valid using linear search.
     * Parameters:
     * - accountNumber: the account number to validate
     * Intended use:
     * - Returns true if accountNumber exists in validAccounts, otherwise
false.
     */
    public boolean isValid(int accountNumber) {

        // Loops through each valid account number.
        for (int i = 0; i < validAccounts.length; i++) { // i is the index
used to search the array.
            // Checks if the current stored account matches the input.
            if (validAccounts[i] == accountNumber) {
                // Returns true immediately if found.
                return true;
            }
        }

        // Returns false if the number was not found.
```

```
        return false;
    }
}

/**
 * ChargeAccountDemo prompts the user to enter an account number and
 * reports whether it is valid using ChargeAccountValidator.
 */
public class ChargeAccountDemo {

    /**
     * Program entry point.
     * Parameters:
     * - args: command-line arguments (unused)
     * Intended use:
     * - Reads an account number from the user with validation and prints
valid or invalid.
     */
    public static void main(String[] args) {

        // input reads user input from the keyboard.
        Scanner input = new Scanner(System.in); // input is used to read
keyboard input.

        // validator checks whether an account number exists in the valid
list.
        ChargeAccountValidator validator = new ChargeAccountValidator(); //
validator performs the validation.

        // accountNumber stores the numeric account number entered by the
user.
        int accountNumber = 0; // accountNumber holds the user's valid
integer input.

        // isInputValid controls the input validation loop.
        boolean isInputValid = false; // isInputValid is true only after a
valid integer is read.

        // Keeps asking until a valid integer is entered.
        while (!isInputValid) {

            // Prompts the user to enter an account number.
            System.out.print("Enter a charge account number: ");

            // Checks whether the next token is an integer.
            if (input.hasNextInt()) {
                accountNumber = input.nextInt();
                isInputValid = true;
            } else {
                // badInput stores the invalid token so we can consume it.
                String badInput = input.next(); // badInput removes the
invalid entry from the scanner.
                System.out.println("Invalid input. Please enter a numeric
charge account number.");
            }
        }
```

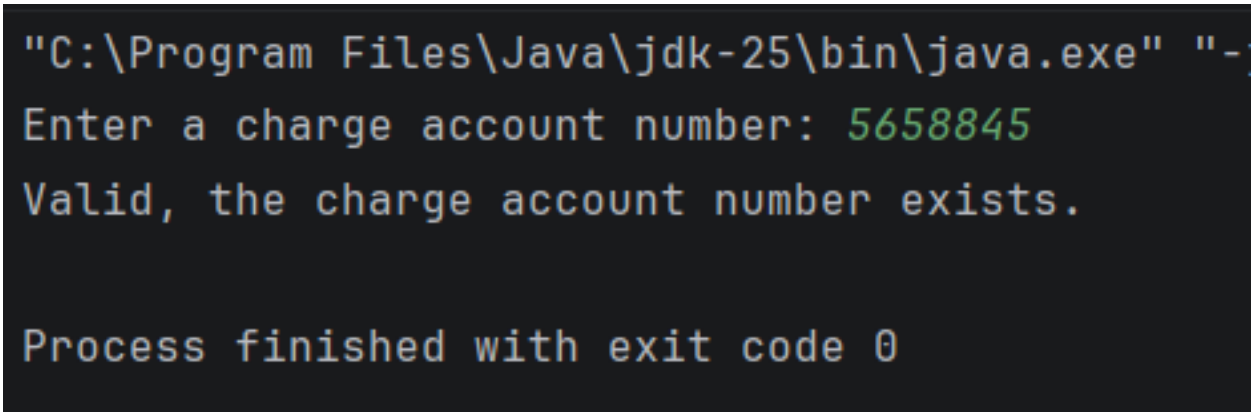
```
        // valid stores whether the account number exists in the list.
        boolean valid = validator.isValid(accountNumber); // valid is true if
accountNumber was found.

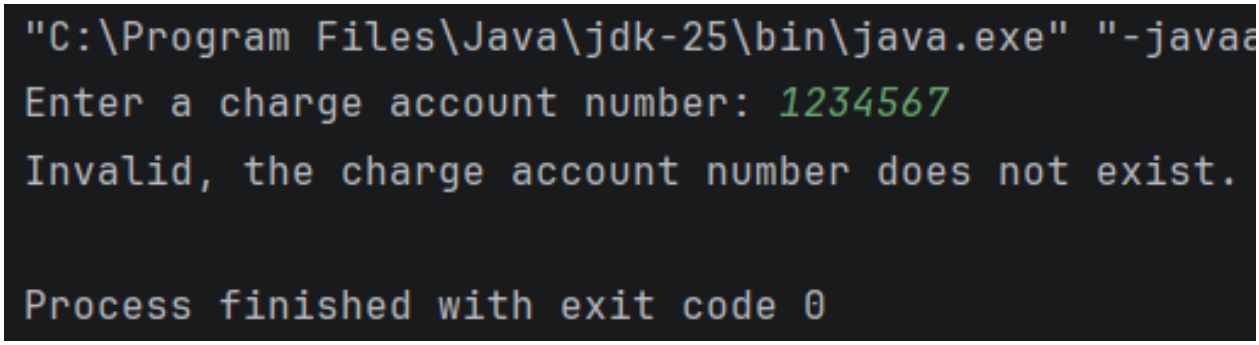
        // Prints the appropriate result message.
        if (valid) {
            System.out.println("Valid, the charge account number exists.");
        } else {
            System.out.println("Invalid, the charge account number does not
exist.");
        }

        // Closes the scanner.
        input.close();
    }
}
```

---

*(Output- screenshots)*



**Problem-3**

*(UML-screenshots)*

